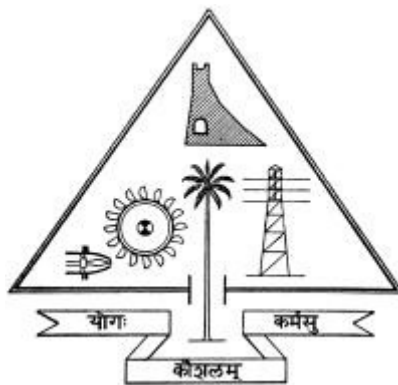


RUSTAUDIT AI: INTRA-PROCEDURAL GRAPH BASED SEMANTIC ANALYSIS AND MULTI-DIMENSIONAL SOFTWARE QUALITY ASSESSMENT FOR RUST

*Mini Project Report submitted in partial fulfillment of the requirements for the
award of the degree of **Master of Computer Applications** of the **APJ Abdul
Kalam Technological University***

Submitted by

**SOORAJ V
(TCR25MCA-2050)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

MARCH 2026

ABSTRACT

Rust has emerged as a prominent systems programming language due to its strong guarantees of memory safety, concurrency, and high-performance execution. Despite these compile-time advantages, the operational quality of Rust applications remains highly dependent on localized developer implementation practices, including fine-grained ownership boundaries, micro-allocation behaviors, and loop safety mechanics. Existing static analysis ecosystems operate primarily on linear token patterns, providing fragmented, rule-based diagnostic alerts devoid of contextual reasoning regarding localized lifecycle mutations. Furthermore, current infrastructure tools lack the cross-layer evaluation models necessary to reason about how a localized memory allocation anti-pattern simultaneously degrades performance and code maintainability indicators inside a single subroutine body.

This paper presents RustAuditAI, an intra-procedural software quality intelligence framework that performs multi-dimensional semantic graph analysis bounded within individual Rust functions. The proposed system parses localized function syntax and constructs deep, interconnected semantic abstractions, including functional ASTs, localized CFGs, and dedicated FLOGs. By executing specialized traversal algorithms across this localized Code Property Graph structure, RustAuditAI unifies structural metrics with semantic analysis across four primary quality vectors: memory safety verification, code execution complexity, localized resource density, and defensive security. These analysis vectors are dynamically synthesized via a weighted valuation model to compute a standardized Rust Quality Index (RQI) representing localized code health. Bounded by the deterministic

structural properties of the generated semantic graphs, an explainable recommendation engine generates context-aware, human-interpretable refactoring suggestions that specify the underlying structural degradation paths and estimate the quantitative execution optimizations achieved through more idiomatic Rust idioms.

CONTENTS

List of Figures	v
List of Abbreviations	v
1 INTRODUCTION	1
2 LITERATURE REVIEW	4
3 ENVIRONMENTAL STUDY	7
3.1 System Configuration	7
3.1.1 Hardware Requirements	7
3.1.2 Software Requirements	8
3.1.3 Software Requirements for Development	8
3.2 Software Specification	9
3.2.1 Python 3.8	10
3.2.2 Rust Toolchain(syn and cargo)	10
3.2.3 The Function-Level Ownership Graph (FLOG) Engine	10
3.2.4 Git	11
3.2.5 NetworkX	11
3.2.6 Explainable AI (XAI) Module	12
3.2.7 Visualization Support	12
4 SYSTEM ANALYSIS	13
4.1 Requirements Analysis	13
4.2 Existing System	15
4.3 Limitations of Existing System	17
4.4 Proposed System	18
4.5 Advantages of Proposed System	20
4.6 Functional Requirements	21
4.6.1 Source Code Input and Upload	21
4.6.2 Language Identification and Preprocessing	22
4.6.3 Graph Abstraction and Representation Generation . .	22
4.6.4 Vulnerability and Quality Vector Detection using LLM	23
4.6.5 Rust Quality Index (RQI) Report Generation	23
4.6.6 Patch Generation, Validation, and Version Management	24
4.6.7 Code Revision and Patch Control	25
4.6.8 Visualization of Subroutine Graph Abstractions	25
4.6.9 Error Handling	25
4.7 Non-Functional Requirements	25
4.7.1 Performance	26
4.7.2 Usability	26
4.7.3 Accuracy and Reliability	26
4.7.4 Security and Confidentiality	27

4.7.5	Maintainability and Extensibility	27
4.7.6	Explainability	27
4.7.7	User-Controlled Patch Management	27
4.8	Feasibility Study	28
4.8.1	Technical Feasibility	28
4.8.2	Operational Feasibility	29
4.8.3	Economical Feasibility	29
Bibliography		31

LIST OF ABBREVIATIONS

AI	– Artificial Intelligence
API	– Application Programming Interface
AST	– Abstract Syntax Tree
BART	– Bidirectional and Auto-Regressive Transformers
BLEU	– Bilingual Evaluation Understudy
CD	– Continous Integration
CI	– Continous Deployment
CSS	– Cascading Style Sheets
DFD	– Data Flow Diagram
GPT	– Generative Pre-trained Transformer
HTML	– HyperText Markup Language
IDE	– Integrated Development Environment
LLM	– Large Language Model
METEOR	– Metric for Evaluation of Translation with Explicit Ordering
Q/A	– Question and Answer
REST API	– Representational State Transfer Application Programming Interface
RoBERTa	– Robustly Optimized BERT Approach
ROUGE	– Recall-Oriented Understudy for Gisting Evaluation
T5	– Text-to-Text Transfer Transformer
URL	– Uniform Resource Locator
UTF-8	– 8-bit Unicode Transformation Format
VS Code	– Visual Studio Code

CHAPTER 1

INTRODUCTION

In the contemporary computing landscape, systems programming languages form the bedrock of mission-critical infrastructure, including operating system kernels, cloud hypervisors, browser engines, and high-performance embedded networks. Historically, these low-level architectural layers were written exclusively in C or C++, exposing them to persistent categories of memory-safety exploitation vectors, such as buffer overflows, use-after-free anomalies, and dangling pointer dereferences. The emergence of the Rust programming language has redefined this domain by enforcing strict compile-time safety guarantees. Through a mathematically rigorous ownership model, affine type systems, and strict borrow checking boundaries, Rust eliminates traditional memory corruption bugs at compile time without the execution overhead of a runtime garbage collector.

However, the compile-time guarantees of Rust do not inherently ensure high-quality, secure, or operationally efficient software. While the compiler prevents fatal undefined behavior, human developers frequently struggle to adapt to the language's demanding structural constraints. This friction leads to widespread structural design flaws and anti-patterns inside individual subroutines. Developers routinely fight the borrow checker by over-relying on expensive runtime allocations, introducing redundant memory movements via excessive `.clone()` invocations, writing high-cyclomatic branching logic, or misusing `unsafe` code blocks to bypass compiler checks for localized optimizations.

In contemporary DevSecOps ecosystems, identifying these quality bottlenecks is severely limited by a fragmented toolchain. Software teams must maintain multiple independent code inspection platforms: running **Clippy** for localized syntactic style lints, executing **cargo-audit** to detect known dependency advisory records, and using separate dynamic profilers to locate execution overhead. This disjointed execution path yields disconnected, plain-text diagnostic outputs that fail to communicate correlations among software dimensions. Traditional lint tools operate through linear token pattern-matching checks, completely lacking the contextual reasoning required to trace how a localized memory micro-allocation flaw simultaneously degrades performance and maintainability scores within a specific function body.

To fundamentally resolve these structural challenges, this project introduces RustAuditAI, a novel, intra-procedural software quality intelligence framework that performs multi-dimensional semantic graph analysis bounded within individual Rust subroutines. Rather than executing shallow line-by-line checks over plain source code, the system ingests localized function syntax and constructs an advanced intermediate abstraction layer. Using the language compiler frontend, the framework builds a unified Code Property Graph (CPG) structure consisting of functional Abstract Syntax Trees (ASTs), localized Control Flow Graphs (CFGs), and a specialized, domain-specific Function-Level Ownership Graph (FLOG) that explicitly traces variable lifecycles, borrows, and structural move mutations.

By executing graph-traversal slicing techniques over this multi-layered architectural representation, RustAuditAI unifies formal software engineering metrics with deep semantic analysis across four core analytical dimensions: memory safety verification, code execution complexity, localized resource density, and defensive security profiling. The quantitative indicators extracted from these dimensions are dynamically evaluated using a weighted linear matrix to compute a standardized Rust Quality Index (RQI)—a transparent,

interpretable metric ranging from 0 to 100 that unifies localized code health into a single operational score.

To bridge the gap between automated detection and developer comprehension, RustAuditAI couples this deterministic graph infrastructure with an explainable artificial intelligence (XAI) interface driven by a Large Language Model (LLM). Bounded strictly by the structural properties of the generated semantic graphs, the system generates context-aware, human-interpretable refactoring insights. Instead of displaying abstract rule flags, the framework outputs detailed natural language reasoning explaining the underlying root cause of structural degradation, mapping the associated data propagation paths, and predicting the exact percentage of performance optimization achieved through idiomatic Rust alternatives. Finally, the system includes a self-contained visual engine that generates side-by-side behavioral diffs and highlighted execution graphs, providing an intuitive, user-controlled patch validation mechanism. By embedding RustAuditAI into software engineering lifecycles, developers can eliminate fragmented lint barriers, enforce rigorous software engineering standards, and maintain optimal performance parameters inside modern systems software.

CHAPTER 2

LITERATURE REVIEW

The evaluation of software quality metrics and vulnerability localization models remains a core research frontier within software engineering and program security ecosystems. Historically, code analysis strategies have been split into two primary areas: rule-based static application security testing (SAST) and learning-based deep representation networks. While each methodology provides standalone analytical value, both exhibit significant design limitations when applied to the unique structural constraints and execution paradigms of the Rust programming language.

Traditional static analysis toolsets, exemplified by industry standard engines such as Fortify and SonarQube, rely heavily on deterministic pattern matching, text regex scanning, and pre-formulated rule databases to capture insecure code constructs. In the context of the Rust programming ecosystem, this paradigm is executed by the official compiler linter, Clippy, which relies on predefined abstract syntax checks to flag unidiomatic expressions. While highly effective at identifying shallow formatting irregularities or basic syntactic variations, these tools fall short when assessing deep logical vulnerabilities or cross-layer behavioral bottlenecks. Because linear token-matching mechanisms lack comprehensive data-flow tracking, they often produce high false-positive rates, miss complex structural defects that span conditional execution pathways, and fail to provide developers with actionable architectural reasoning.

To overcome the challenges of syntax-dependent code checking, compiler research has shifted toward structural graph abstractions that capture program behavior independently of source text. The foundational

breakthrough in this domain was established by Yamaguchi et al. (2014), who formalized the concept of the Code Property Graph (CPG). By merging three distinct compiler representations—the grammatical Abstract Syntax Tree (AST), the execution-path-bound Control Flow Graph (CFG), and the data-dependence-tracking Program Dependence Graph (PDG)—into a single queryable graph database framework, the CPG transformed static analysis into a graph traversal problem. Tools built on this paradigm, such as the open-source analyzer **Joern**, allow security engineers to trace complex data propagation flows from input sources to vulnerable operation sinks. However, these traditional graph-based approaches remain strictly structural and deterministic. Their accuracy depends entirely on manual query development using specialized domain languages, leaving them unable to adapt to novel, context-dependent architectural flaws. Furthermore, classical CPG models are optimized for C/C++ memory models and lack the specialized semantic nodes needed to reason about Rust’s unique ownership, move, and reference borrowing rules.

Concurrently, the integration of deep learning and Natural Language Processing (NLP) has led to the development of massive transformer architectures trained on vast code corpora. Pre-trained language models for code, such as CodeBERT and GraphCodeBERT, have demonstrated strong capabilities across diverse software tasks, including vulnerability classification, defect prediction, and code summarization. Despite these advances, pure deep learning models face severe limitations when operating directly on raw high-level source code. As highlighted by Chakraborty et al. (2022) in their evaluation *"Deep Learning Based Vulnerability Detection: Are We There Yet?"*, token-based language models often lack true compiler-level comprehension. They are highly sensitive to superficial syntax modifications, variable renaming, and coding styles, which leads to unpredictable accuracy drops and high false-negative rates when processing subtle logic errors.

Building upon these foundational advancements, RustAuditAI introduces a highly specialized, intra-procedural software quality intelligence framework

designed to address the unique complexities of the Rust language ecosystem. While existing literature focuses heavily on binary vulnerability detection or syntax-focused parsing, this project shifts the paradigm toward a multi-dimensional quality assessment engine. By extending the classical Code Property Graph with a custom Function-Level Ownership Graph (FLOG), the framework maps Rust's precise variable lifetimes and move metrics into queryable graph spaces. By feeding these structural graph contexts into state-of-the-art reasoning layers, RustAuditAI unifies deterministic software engineering formulas (such as McCabe's Cyclomatic Complexity) with context-aware, explainable AI recommendation streams. This integration delivers an actionable, mathematically grounded, and developer-friendly code intelligence tool tailored for modern secure systems development.

CHAPTER 3

ENVIRONMENTAL STUDY

This chapter outlines the hardware and software configuration environments required to successfully implement, deploy, and execute the RustAuditAI system. It details the platform constraints, compiler front-end parser utilities, language inference tools, and graph visualization libraries that form the underlying stack of the multi-dimensional software quality intelligence pipeline.

3.1 System Configuration

The operational environment for RustAuditAI spans specific physical computing architectures and virtual software execution layers designed to handle high-fidelity abstract syntax parsing and local model inference workflows efficiently.

3.1.1 Hardware Requirements

The hardware requirements outline the minimum resources necessary to operate the system. These resources are crucial for performing code analysis, processing tasks and running machine learning operations.

The key hardware requirements are detailed as follows:

- **Processor:** Intel® Core™ i5 Processor / AMD Ryzen™ 5 Processor or higher, to efficiently handle code processing tasks.
- **Memory:** Minimum 8 GB RAM, to ensure smooth execution of analysis and model operations.

- **Storage:** 50 GB of free disk space, for storing code files and generated outputs.

3.1.2 Software Requirements

The requirements for software specify the necessary platforms, languages and libraries that will be used in running the system. The use of these tools, activities like parsing, integration and documentation become easy. Compatibility assurance is relevant to ensure that the various components of the system communicate effectively.

The key software requirements are detailed as follows:

- **Operating System:** Linux distribution (Ubuntu 22.04 LTS or later recommended), macOS Sequoia, or Microsoft Windows 11 configured with Windows Subsystem for Linux (WSL2).
- **Primary Implementation Platforms:** Python runtime environment version 3.11.x or later, alongside the native Rust compiler toolchain (rustc and cargo engine package utility version ≥ 1.75).
- **Version Control System:** Git open-source distributed source pipeline control suite.
- **Artificial Intelligence Interface Model:** Access token credentials for the centralized Groq inference platform API cluster utilizing the LLaMA-3.3-70B instruction-tuned model parameter footprint.

3.1.3 Software Requirements for Development

Besides the execution specifications, some indispensable are the tools for development, testing and maintenance. They assist in writing and debugging programs, as well as managing the code appropriately. Besides, the tools help in maintaining versions and assembling different systems together.

The key development software requirements for development are detailed as follows:

- **Operating System:** Windows 10, Ubuntu 20.04 LTS or macOS, ensuring cross-platform compatibility.
- **Back End:** Python 3.8 or later, serving as the core implementation language.
- **IDE Used:** Visual Studio Code, Antigravity IDE used for development and debugging tasks.
- **Version Control:** Git open-source distributed source pipeline control suite.
- **Development Tools and Frameworks:**
 - **syn crate:** Specialized parsing framework utilized to read and decode token streams into explicit Rust abstract syntax trees (ASTs).
 - **quote and proc-macro2 crates:** To transform structured AST tokens back into compiler-valid syntactic elements for analysis slicing.
 - **Pydantic crate/library:** Tasked with parsing and structurally enforcing input/output JSON schemas coming from the text reasoning layers.
 - **NetworkX library suite:** Deployed as the core runtime graph data structure layout utility to handle localized nodes, edges, and path traversals for the Function-Level Ownership Graph (FLOG).
 - **Requests or Httpx engine packages:** Configured to manage safe, asynchronous connectivity protocols with cloud API endpoints.
 - **Rich text layout library:** Configured to parse code diagnostics into interactive terminal layout screens.

3.2 Software Specification

The following section provides a detailed description of different software components and technologies that are used to design the system under discussion. This is because each of the software components performs

specific tasks that shall be useful when analyzing the code and preparing the documentation. Each software component is made to perform particular operations that are useful in code analysis and documentation preparation and the use of technologies is meant to ensure a smooth integration process.

3.2.1 Python 3.8

Python functions as the primary execution coordinator for RustAuditAI. It orchestrates the lifecycle of the static analysis pipeline, handling file validation, initiating external compilation checks, managing the graph data structures inside NetworkX, and routing extracted semantic data vectors to the remote inference gateways. Python's mature data structure packages and rich reporting wrappers make it the ideal platform to bind the deterministic compiler graph layers with the downstream explainable text engines.

3.2.2 Rust Toolchain(syn and cargo)

The native Rust ecosystem acts as the foundational parsing source engine for this framework. The syn crate acts as a robust compiler front-end parser that ingests standard .rs file text strings and decodes them into fully structural Abstract Syntax Trees (ASTs). This component preserves localized code expressions, statement mappings, control constructs, and identifier paths. By relying on syn, RustAuditAI gains direct access to precise compiler-level representations of function scopes without requiring fully compiled intermediate binary builds, establishing the raw data foundation for the custom Ownership and Control Flow graphs.

3.2.3 The Function-Level Ownership Graph (FLOG) Engine

The FLOG Engine is a core domain-specific novelty created for this project using the NetworkX library. It translates abstract code definitions into explicit variable lifecycle tracking paths. It generates nodes representing allocation footprints (such as local references, box vectors, and variable re-bindings) interconnected by edges detailing ownership rules (such as

moves, mutable borrows, or immutable aliases). This custom data structure enables the static analysis engine to perform deep intra-procedural lifecycle evaluations, mathematically identifying code flaws like allocation leaks or excessive cloning behaviors before execution.

3.2.4 *Git*

Git is used as the version control system for analyzing the structure of the repository and obtaining historical information regarding the code. This allows the system to work with entire projects by cloning repositories from remote sites, which eliminates manual uploading of separate files. Information at the commit level is also available, for example, for the production of documentation, such as a list of the latest changes.

This would enable the system to be able to identify the different types of changes through commits, thus being able to identify development patterns and get knowledge on how the code is being changed over time. All this would lead to context being better understood, thereby adding a layer of value to the kind of document that is being generated. Other than source code repositories, the system would also allow local source code directories to ensure flexibility with the various types of input sources.

3.2.5 *NetworkX*

It is used to create and analyze the graphs representing the relationships, function calls and dependencies in the code. NetworkX helps the system to model the code structure as a directed graph, where the functions are represented as nodes and the interactions are represented as edges. This allows the system to understand important parts of the program code, such as the starting point, recursion and any dependencies. The use of graph-based analysis increases the understanding of how the code operates. Additionally, the system uses graph analysis to perform complex evaluations of the code, enabling a better understanding of its critical components.

3.2.6 Explainable AI (XAI) Module

The Explainable AI module improves operational transparency by turning raw, numeric quality checks into clear, natural language recommendations. Rather than printing standard compiler diagnostics or raw text warning codes, this component uses structured semantic properties to explain why an expression violates best practices. It explicitly documents the data path where a variable bottleneck emerges and includes structural feedback explaining how a suggested update modifies execution dependencies. This detailed explanation turns abstract graph diagnostics into educational, highly actionable refactoring advice for developers.

3.2.7 Visualization Support

The visualization engine translates internal graph structures into self-contained HTML reports featuring interactive Control Flow Graphs (CFGs) and variable lifecycle grids. Built without external layout software dependencies, it provides custom visual highlights along vulnerable or inefficient execution pathways inside a function body. By showing a side-by-side behavioral comparison between original and proposed code structures, the module enables developers to trace attack vectors or performance bottlenecks visually, improving system transparency and patch management.

CHAPTER 4

SYSTEM ANALYSIS

The practice of performing systems analysis in a technical development scenario focuses on identifying the most effective way to solve a relevant problem—such as detecting vulnerabilities and software quality bottlenecks in source code. Before initiating the development of such a system, project stakeholders, including researchers or student developers, formulate a project proposal and present it to the concerned academic or institutional authority for review and approval.

A structured system inquiry in this context involves evaluating the current challenges developers face in identifying and securing their code, with the goal of enhancing these processes using advanced tools like language parsers and large language models (LLMs). System analysis is used to break down the problem, identify key objectives—such as code transformation, analysis, and vulnerability classification—and ensure all modules work cohesively to deliver accurate and helpful insights to the developer or learner using the system.

4.1 Requirements Analysis

Requirement analysis is a crucial phase in the software development life cycle that focuses on identifying, documenting, and analyzing the functional and technical needs of a system before design and implementation. For the proposed RustAuditAI project, which aims to perform intra-procedural software quality intelligence, vulnerability detection, and explainable patch generation within individual Rust functions, this phase plays a vital role in defining the system's scope and expected behavior.

The primary requirement of the system is to support source code blocks written in the Rust programming language. Individual functions must be parsed into a structured abstract form using compiler-level front-ends, specifically the syn crate ecosystem. This compiler-level syntax extraction enables language-idiom validation and forms the foundation for consistent, intra-procedural data-flow and lifetime tracking without requiring full workspace compilation binaries.

Another key requirement is the integration of Large Language Models for semantic code reasoning and software quality analysis. The system must be capable of extracting localized graph properties from the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Function-Level Ownership Graph (FLOG), and providing them as structured context inputs to LLMs for identifying safety defects, allocation bottlenecks, and logical vulnerabilities. In addition to detection, the system is required to generate natural-language explanations using Explainable AI (XAI) techniques, ensuring that developers understand the root cause, execution flow, and structural impact of each detected quality degradation pattern.

The system must also support automatic patch generation, where secure and optimized code fixes are produced for identified function flaws. These fixes should be presented in a clear old-versus-new diff format to allow developers to review, apply, or refine the suggested patches. Furthermore, visualization support is required to generate interactive, self-contained functional Control Flow Graphs that visually illustrate the propagation of variables, resource allocations, and the specific structural branches leading to code degradation.

In addition to functional requirements, several non-functional requirements must be satisfied. The system should provide highly efficient intra-procedural analysis with rapid execution times suitable for active development loops. It must maintain strict determinism in its mathematical scoring engine and be extensible enough to incorporate additional performance or safety criteria in the future. Usability is also a critical

requirement, with metrics and reports designed to be clear, educational, and developer-friendly.

This requirement analysis ensures that the RustAuditAI system is aligned with its intended objectives and provides a reliable foundation for subsequent system design, implementation, and evaluation.

4.2 Existing System

Existing systems and tools for Rust code verification and vulnerability detection are mainly designed to identify known insecure coding patterns or isolated syntax deviations. Most of these systems rely on traditional static analysis techniques, strict compiler rule sets, or independent linters. While such approaches are effective for enforcing foundational language rules, they often struggle to capture multi-dimensional structural correlations, subtle performance bottlenecks, and logic-level design flaws inside subroutines. Furthermore, many existing tools operate in complete isolation and do not provide a unified framework for measuring overall software quality from an architectural perspective.

Another major limitation of existing systems is their lack of contextual explainability and automated remediation support. Most tools focus only on detection and produce plain-text warnings without clearly explaining the broader architectural cause, execution overhead, or optimization path. As a result, developers must manually interpret the results and implement fixes, which increases debugging effort and code complexity. Some of the widely known systems include:

- **Clippy:** Clippy is the default, official collection of lints for the Rust programming language. It operates by scanning source code files for the presence of known unidiomatic patterns, styling deviations, and minor logical errors. It is lightweight and fast, making it highly suitable for enforcement during standard compilation routines. However, Clippy relies heavily on syntax pattern matching over the compiler's Abstract Syntax Tree, which limits its ability to dynamically correlate disparate

software attributes. This often results in isolated text warnings that lack explanatory depth regarding how a specific variable change cascades to affect performance and maintainability metrics simultaneously. Additionally, Clippy is restricted strictly to syntactic rule validation and does not offer context-aware natural language explanations or automated patch comparison dashboards.

- **cargo-audit:** Developed by the Rust Secure Code Working Group, cargo-audit focuses entirely on supply-chain security and dependency vulnerability scanning. It operates by evaluating a project's dependency manifest (Cargo.lock) against the official Rust Advisory Database to detect crates with publicly disclosed vulnerabilities (CVEs). While essential for dependency management, cargo-audit is blind to the internal code logic written by the developer. It cannot inspect function bodies, analyze variable lifetimes, or track inefficient resource allocation densities within subroutines. Moreover, cargo-audit primarily functions as a binary checker (reporting the presence or absence of vulnerable dependencies) and provides no utility for custom code refactoring, structural visualization, or automated code optimization.
- **Joern:** Joern is an advanced open-source static analysis engine that compiles source code into a Code Property Graph (CPG), blending abstract syntax trees, control-flow graphs, and data-flow graphs into a unified graph structure. This graph abstraction allows for deep semantic queries over program dependencies. However, Joern's accuracy relies entirely on manually formulated queries written in a specialized domain-specific language, creating a steep learning curve and restricting its utility to expert security auditors. Furthermore, Joern's standard graph schema is optimized for C/C++ pointer behaviors and entirely lacks dedicated node types to model Rust's unique compile-time ownership semantics, lifetime variables, and borrow checking restrictions. It also fails to integrate large language models for explainable reasoning or interactive patch generation.

4.3 *Limitations of Existing System*

Though there is access to documentation technology, the existing systems have the following drawbacks:

- **Dependence on Isolated Syntactic Signatures:** Most existing Rust analysis tools rely on rigid, syntax-based signature matching to flag unidiomatic code blocks. These signatures check for precise token patterns and fail to reason about the underlying semantic intent of the subroutine. Consequently, such systems struggle to identify custom logical inefficiencies or context-dependent resource bottlenecks that do not match a pre-configured linter rule.
- **Fragmented and Compartmentalized Outputs:** Current tools operate as completely independent utilities, forcing developers to manage separate workflows for code styling, security checking, and profiling. This fragmentation prevents the cross-layer correlation of data; for instance, an entry-level developer cannot see how an over-reliance on deep memory copies (`.clone()`) to bypass a compiler warning directly degrades performance while simultaneously complicating the function's long-term maintainability.
- **Lack of Standardized Software Quality Metrics:** The existing ecosystem produces discrete lists of diagnostic warnings and errors rather than providing a unified evaluation of overall code health. The absence of a standardized, multi-dimensional metric specific to Rust makes it difficult for project managers and developers to easily measure, track, or prioritize structural improvements over the lifecycle of a subroutine.
- **Absence of Explainable Context and Educational Feedback:** Legacy tools generate brief error messages or static diagnostic links that state what code line is triggering an alert, but omit a comprehensive explanation of why it degrades software quality. This missing reasoning

path slows down verification and triage, offering very little educational value to developers who are trying to learn how to write more idiomatic Rust code.

- **Lack of Automated and Interactive Remediation Support:** Current tools stop at alerting the developer to a potential violation and offer no active assistance in correcting it. The responsibility of researching, writing, and validating an alternative implementation remains entirely manual. This manual remediation process is time-consuming and error-prone, highlighting the clear need for systems that synthesize, display, and apply validated fixes automatically under developer control

4.4 Proposed System

The proposed system, RustAuditAI, is designed to provide a comprehensive, intra-procedural software quality intelligence framework by integrating compiler-level graph abstractions with the advanced reasoning and context-aware capabilities of Large Language Models (LLMs). By combining deterministic program structures with language-tuned reasoning layers, the system achieves highly precise, context-aware, and multi-dimensional quality assessment bounded strictly within individual Rust functions. Unlike conventional linters or rule-based tools that rely on isolated syntax patterns, this system constructs an internal, multi-layered Code Property Graph tailored explicitly for Rust's unique execution semantics, exposing variable states, localized branching structures, and micro-allocation operations in a unified format.

The system translates localized function blocks into structured semantic definitions, creating functional Abstract Syntax Trees (ASTs), localized Control Flow Graphs (CFGs), and a dedicated Function-Level Ownership Graph (FLOG) that explicitly maps variable lifetimes, borrows, and ownership re-bindings. These representations serve as the structured input data for an LLM specialized in code intelligence, enabling the framework to evaluate complex code behaviors that traditional tools miss. The proposed architecture

consists of the following core modules:

- **Syntax Parsing & Graph Generation Module:** This module accepts target Rust function code text and utilizes the compiler frontend syn crate to extract an abstract syntax tree. The module maps localized code expressions, conditional paths, and identifier references into an interconnected graph layer. By standardizing raw text into an active graph structure, it builds the explicit functional blueprint needed for multi-vector inspection.
- **The FLOG Engine Module:** Once the baseline syntax structures are built, this specialized engine generates the Function-Level Ownership Graph. It defines nodes for every localized variable allocation and reference type, connecting them with explicit semantic edges that track lifetime bounds, move variables, and borrow mutations. This maps Rust's precise compiler rules directly into a mathematical graph layout.
- **Multi-Dimensional Metric Computation Engine:** This component simultaneously navigates the generated graphs across four distinct quality vectors: Safety (tracking unsafe blocks and borrow mechanics), Performance (measuring allocation and clone density), Maintainability (calculating McCabe's Cyclomatic Complexity), and Security (identifying hazardous API traits). It applies formal software engineering formulas to produce normalized scores for each vector.
- **The RQI Synthesis Module:** This engine aggregates the independent criteria scores using a weighted valuation matrix to compute the unified Rust Quality Index (RQI) on a scale from 0 to 100. It utilizes a dynamic penalty matrix where a critical safety violation or illegal alias automatically triggers non-linear deductions that lower adjacent quality vectors, providing a true mathematical reflection of localized code health.
- **Explainable AI (XAI) & Patch Synthesis Module:** Upon identifying code flaws, this module feeds the structured graph anomalies into the

LLM interface, bounding its reasoning strictly within the deterministic facts of our graph. The module generates targeted patch options in an old-versus-new diff format alongside clear natural language reasoning. It explains the exact cause of code degradation, outlines the refactoring strategy, and quantifies the expected performance improvements to guide the developer toward optimal programming practices.

4.5 Advantages of Proposed System

The proposed software quality intelligence framework offers several key advantages that enhance the static analysis process and provide significant value to systems developers and educational institutions alike. By combining specialized ownership graphs with bounded LLM reasoning layers, the system ensures highly precise, unified, and actionable code quality evaluations.

- **Unified Multi-Dimensional Evaluation:** Instead of forcing developers to jump between fragmented linter and security binaries, the system evaluates code health across four critical engineering vectors simultaneously, mapping the results onto a singular dashboard.
- **Rust-Specific Ownership Mapping:** The inclusion of a dedicated Function-Level Ownership Graph (FLOG) bridges a massive gap in academic literature by providing an explicit graph structure to track variable moves, lifetimes, and borrow distributions inside function scopes.
- **Mathematical and Deterministic Scoring:** The creation of the Rust Quality Index (RQI) provides a standardized, objective indicator (0–100) grounded in established formulas like Cyclomatic Complexity, allowing team leads to easily track code quality evolution.
- **Explainable, Context-Aware AI Remediation:** By constraining the LLM within the deterministic facts of the semantic graphs, the

system completely prevents AI hallucinations, producing high-fidelity, educational explanations that detail the precise root causes of code degradation.

- **Interactive, User-Controlled Patching:** The framework displays proposed alternative code snippets in a visual side-by-side diff layout, ensuring that all modifications remain fully under developer control and preventing unauthorized codebase revisions.
- **Highly Optimized Performance Loop:** Because the framework is strictly scoped to intra-procedural analysis (evaluating a single function at a time), it circumvents the immense processing bottlenecks of cross-workspace points-to analysis, delivering rapid feedback suitable for agile development workflows.

4.6 Functional Requirements

The proposed system is designed to perform intra-procedural software quality analysis, vulnerability detection, and explainable patch generation within individual Rust functions by leveraging advanced Abstract Syntax Trees (ASTs), localized Control Flow Graphs (CFGs), Function-Level Ownership Graphs (FLOGs), and Large Language Models (LLMs). The functional requirements outline all expected behaviors, user interactions, and internal system capabilities in detail.

4.6.1 Source Code Input and Upload

The system shall allow users to input or upload isolated Rust source code blocks or function snippets through a dedicated interface. Target files can be uploaded or pasted directly as individual .rs files or function code segments. The system shall validate uploaded data to ensure it contains syntactically valid Rust structural declarations before initiating subsequent compilation checks. The system shall provide immediate, descriptive error messages for unsupported file formats, empty payloads, or major syntax compilation

failures. The system shall not accept files with unsupported extensions or text that does not resolve into a standard language function block.

4.6.2 Language Identification and Preprocessing

Upon submission, the system shall automatically detect the language parameters and map the input text directly to the native Rust analysis pipeline. The system shall isolate the structural bounds of the specific subroutine or function scope, filtering out external module variables or global package definitions to enforce strict intra-procedural analysis boundaries. Based on the detected function format, the system shall prepare the code token stream for direct conversion into an intermediate graph abstraction layer using compiler-level parsing crates.

4.6.3 Graph Abstraction and Representation Generation

The system shall successfully generate a multi-layered Code Property Graph abstraction from the function source code using the compiler frontend syn crate backend. The generated graph representation must:

- Extract a functional Abstract Syntax Tree (AST) mapping localized syntax declarations and statement properties.
- Construct a localized Control Flow Graph (CFG) mapping basic block nodes and conditional execution pathways.
- Build a domain-specific Function-Level Ownership Graph (FLOG) that explicitly models variable lifetimes, references, borrows, and move operations.

The system shall verify successful graph construction and report errors if compilation errors or structural anomalies break graph creation boundaries. The system shall not proceed with subsequent LLM analysis routines if initial graph abstraction generation fails.

4.6.4 Vulnerability and Quality Vector Detection using LLM

The system shall parse the constructed semantic graphs to extract localized graph contexts and variable interaction summaries representing internal function behaviors. The system shall submit these graph properties as structured context inputs into the LLM interface to perform semantic reasoning across multiple evaluation layers. The LLM reasoning engine shall identify:

- Unsafe and memory-safety anomalies, including unvalidated pointer arithmetic, unchecked unsafe scopes, or invalid reference lifecycles.
- Performance and resource efficiency defects, including excessive or redundant `.clone()` operations, unnecessary heap allocations, or inefficient iterator tracking loops.
- Maintainability flaws, including overly complex conditional loops, high line-count expressions, or long nested branch layouts.
- Defensive security bugs, including data validation flaws or reliance on deprecated or hazardous API traits.

The system shall classify and tag every identified flaw with a dedicated severity metric (low, medium, high, critical), exact functional statement location, and targeted CWE tracking identification index.

4.6.5 Rust Quality Index (RQI) Report Generation

Once semantic graph processing is completed, the system shall execute its mathematical scoring engines to compile a comprehensive Rust Quality Index (RQI) report. The generated report shall display:

- A unified, composite RQI health score ranging from 0 to 100 based on weighted evaluation matrices.
- Individual score metrics for each of the independent evaluation vectors (Safety, Performance, Maintainability, Security).

- Non-linear deduction logging displaying how severe single anomalies dynamically penalize adjacent vectors.
- Technical descriptions, impact scenarios, and specific mitigation options for every identified flaw.

The completed quality reports shall be exportable as highly readable, self-contained dashboard files or PDF documents for static project review.

4.6.6 Patch Generation, Validation, and Version Management

After code flaws are flagged, the system shall visually highlight the exact functional statement lines inside the workspace interface. The system shall invoke the LLM-based patch synthesis engine to output one or multiple secure, compilable refactoring candidates for the identified vulnerabilities. Each generated patch recommendation must include:

- An optimized, refactored function source code snippet replacing the original vulnerable block.
- Natural language explanations detailing the structural mechanics of the fix.
- Estimated performance improvements, resource savings, or index optimization impacts.

The system shall present all refactoring recommendations to the user in a side-by-side comparison format, ensuring that code updates remain completely user-controlled. Upon user confirmation and approval of a recommended patch, The original function code block is replaced by the secure version in the current workspace. The framework automatically switches to the patched version as the active codebase. The system regenerates the corresponding AST, CFG, and FLOG graph models to verify that the target issue has been successfully resolved.

4.6.7 Code Revision and Patch Control

The system shall maintain a thorough intra-procedural revision database tracking every approved modification layer. Each revision entry shall store a full historical copy of the preceding code structure, the specific patch details, and an exact timestamp identifier. The system shall provide an accessible rollback/revert feature, enabling users to instantly reset the function back to any earlier version state. Upon executing a rollback, the function code is immediately restored, and its original graph abstractions and metrics are re-calculated and displayed.

4.6.8 Visualization of Subroutine Graph Abstractions

The system shall provide a built-in visualization interface displaying high-fidelity interactive models of the Control Flow Graph (CFG) and the Function-Level Ownership Graph (FLOG). During the initial assessment phase, inefficient branching execution paths, excessive allocation nodes, and vulnerable instruction sequences shall be highlighted inside the graph interface. During patch validation, the interface shall render side-by-side graph models showing the old structure vs. the new optimized abstraction layout to visually demonstrate path removal and optimization.

4.6.9 Error Handling

The system shall provide meaningful error messages for validation failures, unsupported syntax, parser errors, or timeout exceptions during API communication. The framework shall handle complex, large subroutines gracefully, logging nested parsing exceptions without breaking the workspace database or corrupting active data states.

4.7 Non-Functional Requirements

The non-functional requirements specify quality attributes for the system. They ensure that the system's output is accurate, consistent and that its functions

effectively and remains user-friendly. The non-functional requirements specify the quality attributes for the system to ensure that it is effective, user-friendly and produces the right output and that it emphasizes on scalable, usability and reliability that underpins implementing a robust and practical AI-powered documentation solution.

4.7.1 Performance

The system must execute all operational tasks—including syntax tree generation, FLOG construction, metrics extraction, LLM context evaluation, and report compilation—efficiently. Because the platform is strictly restricted to an intra-procedural execution scope, response latencies during static analysis loops should remain low enough to facilitate an interactive development loop without performance bottlenecks.

4.7.2 Usability

The user interface should prioritize clarity, accessibility, and clean design for both learners and developers. The dashboard must present quality indicators clearly, utilizing interactive code panels, synchronized side-by-side patch diff layouts, and visual graph displays that make complex ownership mutations and refactoring steps easy to trace.

4.7.3 Accuracy and Reliability

System reliability is maintained through rigorous, deterministic graph metrics combined with transparent patch validation loops. The framework must accurately evaluate structural and control flows, ensuring that the calculated metrics reflect the true state of the function scope. The platform preserves source stability by ensuring that no code modifications are ever executed automatically without explicit developer consent.

4.7.4 *Security and Confidentiality*

All user-submitted function code blocks are analyzed in an isolated, secure execution environment. The platform ensures that no uploaded source material or intermediate graph databases are permanently stored or exposed after analysis is complete. Proper input validation and sandboxing are enforced throughout the pipeline to protect the tool from malicious execution exploits.

4.7.5 *Maintainability and Extensibility*

The framework must feature a highly modular architecture that cleanly separates the parsing front-end, the metric calculation module, the LLM reasoning interface, and the reporting dashboard. This modular configuration allows developers to easily update assessment rules, swap underlying text inference models, or integrate the framework into continuous deployment workflows without requiring an architectural overhaul.

4.7.6 *Explainability*

All code quality evaluations and vulnerability detections must be justified with natural language explanations detailing the underlying structural root causes. Every suggested patch must include structured reasoning that clearly defines the safety or performance enhancements achieved, allowing developers to understand both the cause of the issue and the exact structural effect of the proposed solution.

4.7.7 *User-Controlled Patch Management*

The application must guarantee that patch validation remains entirely under user control. Developers must be provided with side-by-side textual code comparisons and structural graph differences, enabling them to evaluate, accept, or discard recommendations as needed before any modifications are committed to the file system.

4.8 Feasibility Study

A feasibility study examines a proposed system to determine its viability, impact on operations, ability to meet user needs, and cost-effectiveness. While any project might seem achievable given unlimited time and resources, it is essential to assess feasibility early to ensure practicality and success. Systems that integrate emerging technologies, such as advanced language compilers and large language models, require such evaluation to anticipate implementation challenges, technical risks, and operational constraints. In the case of the proposed RustAuditAI framework, a feasibility study helps evaluate whether the intra-procedural analysis system can fulfill technical metrics, work efficiently within target workflows, and deliver practical value for software quality engineering. The study is divided into three sections:

4.8.1 Technical Feasibility

The technical feasibility of RustAuditAI is grounded in the architectural choice to restrict code parsing to an intra-procedural scope using mature compiler utilities. By utilizing the robust `syn` crate, the frontend can reliably transform raw function text into structured Abstract Syntax Trees without the immense compute overhead required for full cross-workspace compilation. The integration of the `NetworkX` library allows for high-performance generation and traversal of the custom Function-Level Ownership Graph (FLOG), accurately tracking variables, lifetimes, and borrows. The language reasoning layers are handled via secure API connections to highly optimized instruction-tuned models like LLaMA-3.3-70B, which provide fast text inference speeds and low latency. The local visualization engines utilize clean HTML and CSS formatting, avoiding dependencies on heavyweight external layout frameworks. Because all underlying parsing libraries, graph models, and API interfaces are well-supported and extensively documented, the construction of the system is highly feasible within standard computing environments without specialized hardware acceleration.

4.8.2 Operational Feasibility

Operational feasibility evaluates whether the system can function efficiently in the intended environment and satisfy user requirements. RustAuditAI is explicitly structured to introduce minimal disruption into active development loops, operating as a lightweight, interactive function-level assistant. By focusing on localized routines, it provides immediate quality metrics, RQI logs, and visual graph trees that help developers rapidly inspect code logic. The split dashboard—showing side-by-side code diffs, actionable explanations, and patch histories—is designed to be highly accessible to developers with varying security and systems expertise. The platform is equally effective for project managers tracking quality trends and for students learning idiomatic systems programming concepts. Therefore, the tool is highly feasible from an operational perspective across both educational and industry environments.

4.8.3 Economical Feasibility

Economic feasibility assesses whether the benefits of the system justify the associated costs. The primary development expenditures involve minor computing resources for local graph generation and processing standard cloud API tokens for text reasoning queries. Relying on open-source compiler crates like syn and standard hardware configurations drastically lowers upfront infrastructure costs compared to building proprietary analysis engines. By automatically evaluating software metrics, flagging allocation vulnerabilities, and providing instant refactoring patches under developer control, the framework significantly reduces the time and cost associated with manual code optimization and security code remediation. This automated approach delivers long-term productivity gains and reduces potential runtime vulnerabilities, making RustAuditAI highly cost-effective and economically feasible for academic research and deployment pipelines.

BIBLIOGRAPHY

- [1] J. Song, Z. Zhang, Z. Tang, S. Feng, and Y. Gu, "Improving Code Summarization With Tree Transformer Enhanced by Position-Related Syntax Complement," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 9, pp. 4776–478X, Sept. 2024, doi: 10.1109/TAI.2024.3395231.
- [2] M. F. Rozi, T. Ban, S. Ozawa, A. Yamada, T. Takahashi, S. Kim, and D. Inoue, "Detecting Malicious JavaScript Using Structure-Based Analysis of Graph Representation," *IEEE Access*, vol. 11, pp. 102727–1027XX, 2023, doi: 10.1109/ACCESS.2023.3317266.
- [3] M. M. Ghahfarokhi, A. Khademian, S. Kianiangelafshani, A. Asadi, H. Jahantigh, and A. Heydarnoori, "Beyond Syntax: Unleashing the Power of Computational Notebooks Code Metrics in Documentation Generation," in *Proc. IEEE/ACM Int. Conf. AI Engineering – Software Engineering for AI (CAIN)*, Lisbon, Portugal, Apr. 2024, pp. 278–280, doi: 10.1145/3644815.3644979.
- [4] W. Wang, Y. Zhang, Y. Sui, Y. Wan, Z. Zhao, J. Wu, P. S. Yu, and G. Xu, "Reinforcement-Learning-Guided Source Code Summarization using Hierarchical Attention," *IEEE Transactions on Software Engineering*, 2020.
- [5] S. Yao, Y. Shen, D. Liu, and J. Hu, "CodeDocAgent: Leveraging Large Language Models for Accurate and Contextual Code Documentation," in *Proceedings of IEEE*, 2025.
- [6] Y. Zhu, X. Zhao, X. Li, Y. Zou, H. Yuan, Y. Wang, and B. Xie, "RepoSummary: Feature-Oriented Summarization and Documentation Generation for Code Repositories," in *Proc. ACM Conf.*, 2025.

- [7] J. Y. Khan and G. Uddin, “Combining Contexts from Multiple Sources for Documentation-Specific Code Example Generation,” in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2023, pp. 683–693, doi: 10.1109/SANER56733.2023.00071.
- [8] S. R. Thota, S. Arora, and S. Gupta, “AI-Driven Automated Software Documentation Generation for Enhanced Development Productivity,” in *Proc. Int. Conf. Data Science and Network Security (ICDSNS)*, 2024.
- [9] D. P. Ghale and M. Dabbagh, “Automated Code Comments Generation Using Large Language Models: Empirical Evaluation of T5 and BART,” *IEEE Access*, vol. 13, pp. 141420–1414XX, 2025, doi: 10.1109/ACCESS.2025.3597601.
- [10] T. T. Procko and S. Collins, “Automatic Code Documentation with Syntax Trees and GPT: Alleviating Software Development’s Most Redundant Task,” Embry-Riddle Aeronautical University, Daytona Beach, FL, USA.